

Question

4. Write a Java Program to implement the given class diagram with Customer, Order, SpecialOrder, and NormalOrder classes.

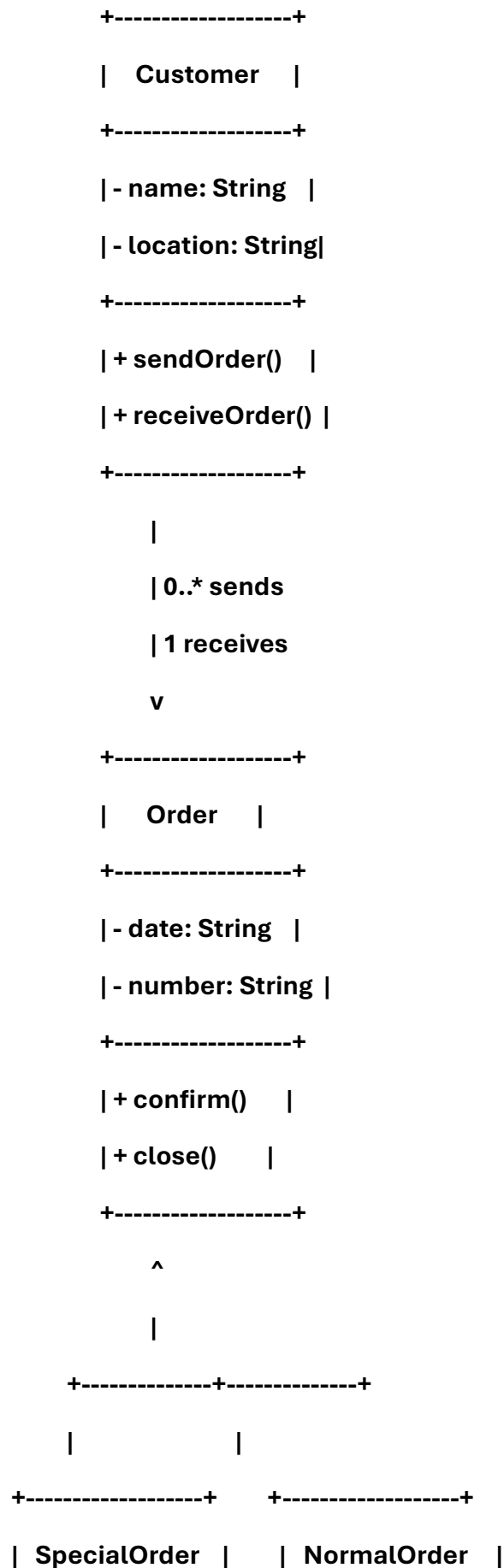
Steps for Execution

1. Open Eclipse IDE (or any Java IDE).
2. Create a new Java project named WeekFourAssignment.
3. Inside src, create a file named weekfouruml.java.
4. Copy the program code into the file.
5. Save the file.
6. Compile and run the program.
7. Observe the output in the console.

Algorithm

1. Define a Customer class with attributes name and location, and methods sendOrder() and receiveOrder().
2. Define a base Order class with attributes date and number, and methods confirm() and close().
3. Create SpecialOrder class inheriting from Order, adding method dispatch().
4. Create NormalOrder class inheriting from Order, adding methods dispatch() and receive().
5. In the weekfouruml class:
 - Create a Customer object.
 - Create SpecialOrder and NormalOrder objects.
 - Call methods to demonstrate sending, confirming, dispatching, receiving, and closing orders.
6. Display results on the console.

UML DIAGRAM:



+-----+	+-----+
- date: String	- date: String
- number: String	- number: String
+-----+	+-----+
+ confirm()	+ confirm()
+ close()	+ close()
+ dispatch()	+ dispatch()
	+ receive()
+-----+	+-----+

CODE:

```
// Customer class
```

```
class Customer {
```

```
    private String name;
```

```
    private String location;
```

```
    public Customer(String name, String location) {
```

```
        this.name = name;
```

```
        this.location = location;
```

```
    }
```

```
    public void sendOrder(Order order) {
```

```
        System.out.println(name + " from " + location + " is sending order " +
order.getNumber());
```

```
    }
```

```
    public void receiveOrder(Order order) {
```

```
        System.out.println(name + " received order " + order.getNumber());
```

```
}  
}
```

```
// Base Order class
```

```
class Order {
```

```
    private String date;
```

```
    private String number;
```

```
    public Order(String date, String number) {
```

```
        this.date = date;
```

```
        this.number = number;
```

```
    }
```

```
    public void confirm() {
```

```
        System.out.println("Order " + number + " confirmed.");
```

```
    }
```

```
    public void close() {
```

```
        System.out.println("Order " + number + " closed.");
```

```
    }
```

```
    public String getNumber() {
```

```
        return number;
```

```
    }
```

```
}
```

```
// SpecialOrder inherits from Order
```

```
class SpecialOrder extends Order {
```

```

    public SpecialOrder(String date, String number) {
        super(date, number);
    }

    public void dispatch() {
        System.out.println("Special order " + getNumber() + " dispatched.");
    }
}

// NormalOrder inherits from Order
class NormalOrder extends Order {
    public NormalOrder(String date, String number) {
        super(date, number);
    }

    public void dispatch() {
        System.out.println("Normal order " + getNumber() + " dispatched.");
    }

    public void receive() {
        System.out.println("Normal order " + getNumber() + " received.");
    }
}

// Main class renamed as WeekFouruml
public class WeekFouruml {
    public static void main(String[] args) {
        Customer c1 = new Customer("Haarika", "Visakhapatnam");
    }
}

```

```
SpecialOrder so = new SpecialOrder("2026-08-10", "SO123");
NormalOrder no = new NormalOrder("2026-08-10", "NO456");

c1.sendOrder(so);
so.confirm();
so.dispatch();
so.close();

c1.sendOrder(no);
no.confirm();
no.dispatch();
no.receive();
no.close();

c1.receiveOrder(no);
}
}
```

Output Screenshot

Haarika from Visakhapatnam is sending order SO123

Order SO123 confirmed.

Special order SO123 dispatched.

Order SO123 closed.

Haarika from Visakhapatnam is sending order NO456

Order NO456 confirmed.

Normal order NO456 dispatched.

Normal order NO456 received.

Order NO456 closed.

Haarika received order NO456